

README — Progetto “Support Agent con RAG” (Python + OpenAI)

Questo progetto implementa un mini agente Customer Care con RAG (Retrieval-Augmented Generation).

L'agente:

1. Riceve un messaggio cliente.
2. Lo classifica (tipo richiesta + urgenza).
3. Recupera le informazioni più rilevanti da una knowledge base locale (kb/knowledge.json).
4. Genera una bozza di risposta usando esclusivamente il contesto recuperato.
5. Salva un report completo dell'esecuzione nella cartella runs/.

L'obiettivo è mostrare in modo didattico come costruire un sistema AI modulare con:

- Classificazione
- Retrieval (RAG)
- Guardrail
- Reporting
- Separazione dei moduli

Struttura del Progetto

```
progetto/
├── src/
│   ├── main.py
│   ├── agent.py
│   ├── classifier.py
│   ├── retrieval.py
│   ├── draft_reply.py
│   ├── reporting.py
│   ├── prompts.py
│   ├── api_client.py
│   ├── kb/
│   │   └── knowledge.json
│   └── runs/
└── key.env
```

- kb/ → contiene la knowledge base (manuale interno dell'hotel)
- runs/ → contiene i report generati ad ogni esecuzione
- key.env → contiene la chiave OpenAI

Requisiti

Python 3.10+ consigliato

Installare le dipendenze:

```
pip install openai python-dotenv numpy
```

Creare il file key.env con:

OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx

Come si esegue

Entrare nella cartella src/ e lanciare:

python main.py

Comparirà:

Incolla il messaggio cliente (riga vuota per terminare):
Inserire il messaggio e premere INVIO su riga vuota.

Workflow Completo (Cosa Succede Internamente)

1) Input utente (main.py)

- Legge il messaggio cliente.
- Istanza SupportAgent.
- Chiama agent.run(messaggio).
- Stampa la risposta.
- Mostra i file generati.

main.py è il punto di ingresso del sistema.

2) Orchestrazione (agent.py)

Classe principale: SupportAgent

Metodo chiave: run(user_message)

Esegue in sequenza:

1. Classificazione (classifier.py)
2. Retrieval / RAG (retrieval.py)
3. Generazione risposta (draft_reply.py)
4. Salvataggio report (reporting.py)

Restituisce un oggetto AgentResult contenente:

- label
- urgency
- summary
- context usato
- reply generata
- meta (top_k, scores, file salvati)

3) Classificazione (classifier.py)

Funzione: classify_message(message)

- Invia un prompt al modello AI.
- Richiede risposta JSON con:
 - label
 - urgency
 - summary

Esempio output:

```
{  
  "label": "CHECKIN",  
  "urgency": "MEDIA",  
  "summary": "Chiede informazioni su late check-in."  
}
```

Se l'AI fallisce, viene usato un classificatore rule-based (fallback offline).

4) Retrieval / RAG (retrieval.py)

Funzione: `retrieve_context(query, top_k=3)`

Quando interviene?

Subito dopo la classificazione.

Cosa fa?

1. Carica kb/knowledge.json
2. Calcola embedding dei documenti
3. Calcola embedding della query
4. Calcola similarity (dot product)
5. Seleziona i top_k documenti più rilevanti
6. Costruisce il contesto finale

Il contesto è composto esclusivamente da testi presenti nella knowledge base.

Se una informazione non è in knowledge.json, il sistema non la conosce.

Questo è il cuore del RAG.

5) Generazione risposta (draft_reply.py)

Funzione: `draft_reply(...)`

Costruisce un prompt con regole:

- Usa SOLO il contesto.
- Se il contesto non contiene la risposta, scrivi ESATTAMENTE:
Non ho informazioni sufficienti.

Questo è un guardrail anti-hallucination.

Il modello genera la risposta finale in tono professionale.

6) Reporting (reporting.py)

Funzione: `save_run_report(...)`

Ad ogni esecuzione crea in `runs/`:

1. `run_TIMESTAMP.json`
2. `reply_TIMESTAMP.txt`

Il JSON contiene:

- Messaggio utente
- Classification
- Context usato
- Reply generata
- Metadati retrieval

Serve per:

- Debug
- Analisi
- Dimostrazione didattica

7) Wrapper OpenAI (api_client.py)

Centralizza:

- chat()
- embed()

Gestisce la lettura della chiave API da key.env.

8) Prompt Templates (prompts.py)

Contiene:

- build_classifier_prompt()
- build_reply_prompt()

Serve per:

- Centralizzare le istruzioni AI
- Modificare facilmente comportamento e guardrail

Quando Interviene il RAG

Il RAG interviene nel modulo retrieval.py.

Recupera informazioni esclusivamente da:

kb/knowledge.json

La risposta finale viene generata usando solo quel contesto.

Se modifichi la knowledge base, la risposta cambia.

Questo dimostra che il sistema non è hardcoded.

Esempi di Test

Test in contesto:

A che ora è la colazione?

Posso fare check-in alle 01:30?

Posso portare un cane?

Test fuori contesto:

Avete un campo da tennis?

Risposta attesa:

Non ho informazioni sufficienti.

Cosa Dimostra Questo Progetto

- Modularità
- Separazione delle responsabilità
- Integrazione OpenAI
- Retrieval con embedding
- Guardrail anti-hallucination
- Logging e tracciabilità
- Architettura pronta per evoluzione

Possibili Estensioni Future

- Memoria conversazionale
- Cache embedding
- Filtri per categoria
- API REST con FastAPI
- Sistema multi-turn
- Escalation automatica

Riassunto Finale

Il progetto implementa un agente AI che:

- Comprende il messaggio cliente
- Recupera informazioni da una base dati interna
- Genera una risposta controllata
- Registra tutto per analisi

È un esempio completo e didattico di applicazione RAG reale in Python.